

Practical 13

Aim

To develop a Student Management System using ReactJS and Spring Boot that allows adding and viewing MCA student records through RESTful APIs and database connectivity.

Learning Objectives

1. To understand the architecture of full-stack web applications.
2. To learn how to create REST APIs using Spring Boot.
3. To understand CRUD operations using Spring Data JPA.
4. To learn how to integrate ReactJS frontend with Spring Boot backend.
5. To understand state management using React Hooks.
6. To learn asynchronous data fetching using Fetch API.
7. To understand database operations using JPA and MySQL.
8. To create responsive and interactive user interfaces using ReactJS.
9. To learn JSON data exchange between frontend and backend.
10. To understand component rendering and dynamic data handling in ReactJS.

Software Requirements

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Hard Disk: 20 GB free space

Software Requirements

- Operating System: Windows/Linux/Mac
- Java JDK 17 or above
- Spring Boot Framework
- ReactJS
- Node.js and npm
- MySQL Database
- Maven
- IntelliJ IDEA / Eclipse / VS Code
- Web Browser (Chrome/Firefox)

Input:

McaApplication.java

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

Python Programming Lab – MCA Sem II

```
@SpringBootApplication
public class McaApplication {
    public static void main(String[] args) {
        SpringApplication.run(McaApplication.class, args);
    }
}
```

McaStudentController.java

```
package com.saloni.mca.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.saloni.mca.model.McaStudent;
import com.saloni.mca.repository.StudentRepository;
```

```
@RestController
@RequestMapping("/McaStudents")
@CrossOrigin(origins = "http://localhost:3000")
public class McaStudentController {

    @Autowired
    StudentRepository repo;

    @PostMapping
    public String addStudent(@RequestBody McaStudent s) {
        repo.save(s);
        return "Success";
    }

    @GetMapping
    public List<McaStudent> viewAll() {
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public McaStudent viewById(@PathVariable int id) {
        return repo.findById(id).orElse(null);
    }
}
```

McaStudent.java

```
package com.saloni.mca.model;
```

Python Programming Lab – MCA Sem II

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
```

@Entity

```
public class McaStudent {
```

@Id

```
private int id;
```

```
private String name;
```

```
public int getId() {
```

```
    return id;
```

```
}
```

```
public void setId(int id) {
```

```
    this.id = id;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public void setName(String name) {
```

```
    this.name = name;
```

```
}
```

```
}
```

StudentRepository.java

```
package com.saloni.mca.repository;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import com.saloni.mca.model.McaStudent;
```

```
public interface StudentRepository extends JpaRepository<McaStudent, Integer> {
```

```
}
```

ReactJS

app.js

```
import React, { useEffect, useState } from "react";
```

```
import "./App.css";
```

```
function App() {
```

```
    const [students, setStudents] = useState([]);
```

```
    const [id, setId] = useState("");
```

Python Programming Lab – MCA Sem II

```
const [name, setName] = useState("");
const [loading, setLoading] = useState(true);

useEffect(() => {
  fetch("http://localhost:8080/McaStudents")
    .then((res) => res.json())
    .then((data) => {
      setStudents(data);
      setLoading(false);
    })
    .catch((err) => {
      console.error("Error fetching students:", err);
      setLoading(false);
    });
}, []);

const addStudent = () => {
  if (!id || !name) {
    alert("Please enter both ID and Name");
    return;
  }

  fetch("http://localhost:8080/McaStudents", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      id: Number(id),
      name: name,
    }),
  })
    .then((res) => res.text())
    .then((msg) => {
      alert(msg);
      window.location.reload();
    })
    .catch((err) => {
      console.error("Error adding student:", err);
      alert("Failed to add student");
    });
};

return (
```

```
<div className="container">
  <div className="glass-card">
    <header>
      <h1>MCA Students</h1>
      <p className="subtitle">Management System</p>
    </header>

    <div className="form-section">
      <div className="input-group">
        <input
          type="number"
          placeholder="Student ID"
          value={id}
          onChange={(e) => setId(e.target.value)}
          id="student-id"
        />
        <label htmlFor="student-id">ID</label>
      </div>

      <div className="input-group">
        <input
          type="text"
          placeholder="Student Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          id="student-name"
        />
        <label htmlFor="student-name">Name</label>
      </div>

      <button className="add-btn" onClick={addStudent}>
        <span>Add Student</span>
      </button>
    </div>

    <hr className="divider" />

    <div className="list-section">
      <h2>Student Directory</h2>
      {loading ? (
        <div className="loader">Loading...</div>
      ) : students.length > 0 ? (
        <ul className="student-list">
          {students.map((s) => (
```

```
        <li key={s.id} className="student-item">
          <span className="sid">#{s.id}</span>
          <span className="sname">{s.name}</span>
        </li>
      )}
    </ul>
  ) : (
    <p className="empty-msg">No students found.</p>
  )}
</div>
</div>
</div>
);
}
```

export default App;

Output:

